

Coding School

Dispatchers in Kotlin Coroutines

Coding School

Dispatchers help Coroutines in deciding the thread on which the task has to be done. We use Coroutines to perform certain tasks efficiently.

Coroutines run the task on a particular thread. This is where the Dispatchers come into play.

Coroutines take the help of Dispatchers in deciding the thread on which the task has to be done.

Dispatchers in Kotlin Coroutines are like Schedulers in RxJava.

Coding School

There are 4 types of Dispatchers in Kotlin Coroutines:

- **Dispatchers.Default**
- **Dispatchers.IO**
- **Dispatchers.Main**
- **Dispatchers.Unconfined**

Coding School

Dispatchers.Default

We should use `Dispatchers.Default` to perform CPU-intensive tasks.

Example use cases:

- Doing heavy calculations like Matrix multiplications.
- Doing any operations on a bigger list present in the memory like sorting, filtering, searching, etc.
- Applying the filter on the Bitmap present in the memory, NOT by reading the image file present on the disk.
- Parsing the JSON available in the memory, NOT by reading the JSON file present on the disk.
- Scaling the bitmap already present in the memory, NOT by reading the image file present on the disk.
- Any operations on the bitmap that are already present in the memory, NOT by reading the image file present on the disk.

So, now we know where we can use the `Dispatchers.Default`.

We can think of `Dispatchers.Default` as a `Schedulers.computation()` of `RxJava`.

```
launch(Dispatchers.Default) {  
    // Your CPU-intensive task  
}
```

Coding School

Dispatchers.IO

We should use `Dispatchers.IO` to perform disk or network I/O-related tasks.

Example use cases:

- Any network operations like making a network call.
- Downloading a file from the server.
- Moving a file from one location to another on disk.
- Reading from a file.
- Writing to a file.
- Making a database query.
- Loading the Shared Preferences.

So, now we know where we can use the `Dispatchers.IO`.

In a nutshell, anything related to file systems or networking should be done using `Dispatchers.IO` as those tasks are IO-related tasks.

We can think of `Dispatchers.IO` as a `Schedulers.io()` of `RxJava`.

```
launch(Dispatchers.IO) {  
    // Your IO related task  
}
```

Coding School

Dispatchers.Main

We should use `Dispatchers.Main` to run a coroutine on the main thread of Android. We all know where we use the main thread of Android. Mainly at the places where we interact with the UI and perform small tasks.

Example use cases:

- Performing UI-related tasks.
- Any small tasks like any operations on a smaller list present in the memory like sorting, filtering, searching, etc.

So, now we know where we can use the `Dispatchers.Main`.

We can think of `Dispatchers.Main` as a `AndroidSchedulers.mainThread()` of `RxAndroid` (RxJava bindings for Android).

```
launch(Dispatchers.Main) {  
    // Your main thread related task  
}
```

Coding School

Dispatchers.Unconfined

"As per the official documentation: A coroutine dispatcher that is not confined to any specific thread. It executes the initial continuation of a coroutine in the current call frame and lets the coroutine resume in whatever thread that is used by the corresponding suspending function, without mandating any specific threading policy."

It does not change the thread. When it is started, it runs on the thread on which it was started. If it is resumed, it runs on the thread that resumed it.

If I put it simply: We should use `Dispatchers.Unconfined` when we do not care where the coroutine will be executed.

```
launch(Dispatchers.Unconfined) {  
    // Your task for which you do not care about the thread on w  
}
```

Example use case:

- Unit Testing as `Dispatchers.Unconfined` does not change the thread.